

CPNAV: Calibrated Vision–Language Navigation for Object-Goal Search in Unseen Environments

Qizhao Chen¹, Yuanhong Zeng², Shoh Nishino¹, and Anushri Dixit¹

Abstract—Directing a robot to locate a target object in unseen environments remains a central challenge in embodied AI. Vision–Language Model (VLM)-based methods provide rich semantic reasoning but often produce uncalibrated and overconfident action decisions. We introduce CPNAV, a navigation framework that combines collision-aware planning with statistically calibrated VLM action scoring for map-free object-goal navigation. From RGB-D input, CPNAV constructs a navigability map, generates collision-free motion primitives, and queries two VLMs for goal detection and action scoring. The resulting scores are calibrated offline via conformal prediction (CP) using oracle actions obtained through a tree-structured exploration procedure with backtracking, providing finite-sample guarantees at a user-specified risk level. The CP layer is model-agnostic and can wrap any VLM policy. Online, calibrated VLM scores guide action selection, while a tree-structured exploration history records alternative actions and allows backtracking. We evaluate CPNAV in both simulation and real-world hardware settings. In AI Habitat simulation, compared with the uncalibrated baseline method, CPNAV achieves relative SR and SPL improvements of 6.2% and 11.4% on HM3D v0.2, and 12.4% and 8.8% on MP3D, respectively. In real-world deployment on a modified Hiwonder MentorPi robot, the corresponding improvements in SR and SPL are 24.9% and 15.2%, respectively.

I. INTRODUCTION

Robotic navigation in complex unseen indoor environments remains challenging because effective decision-making must tightly couple low-level path planning with high-level semantic understanding, while generalizing to novel layouts without prior maps or goal location information [1]. Traditional rule-based planners can handle geometric obstacle avoidance but rely on hand-tuned rules and expert knowledge, making them brittle in unforeseen situations [2]. In contrast, learning-based navigation planners, particularly those based on reinforcement learning (RL), often require painstaking reward shaping and large-scale training data [3]. Such RL-based planning approaches still struggle to generalize across diverse and unseen environments [4].

A key limitation of both rule-based and purely learning-based navigation methods is the absence of semantic understanding, which is essential for efficient goal-directed behavior in realistic environments. Object-goal navigation benefits from reasoning about object-object and object-scene relationships and from leveraging commonsense priors about

given environment layouts. By integrating these semantic priors, the robotic agent can effectively constrain the search process, significantly reducing the branching factor of the search tree and prioritizing paths that align with contextual expectations. Semantic information and common sense reasoning also allow the agent to infer the probable locations of goal objects even when they are not immediately present within the agent’s current field of view [5]. Recent Vision-Language Models (VLMs) such as Gemini [6], LLaVA [7], and GPT [8] enable injection of commonsense and scene priors into navigation, bridging geometric perception and high-level planning [9]. Through large-scale multimodal pretraining, these models acquire semantic priors that support generalization to previously unseen environments, reducing reliance on environment-specific training [10], [11].

Although pretrained VLMs supply strong semantic cues, their internal heuristics, which are the learned rules that guide their behavior, are not calibrated for sequential decision making [12]. VLM outputs are optimized for token prediction or image-text alignment rather than long-horizon action selection under physical constraints. As a result, they often assign non-trivial confidence to unsafe, redundant, or goal-irrelevant actions [13], [14]. In this work, we tackle the problem of unreliable high-level action scoring in unseen environments. Without an explicit mechanism to quantify and control confidence, VLM-driven navigation can assign non-trivial scores to semantically plausible but suboptimal or unsafe actions. Over long horizons, such miscalibration can manifest as inefficient exploration, excessive backtracking, and action loops.

Conformal Prediction (CP) provides a principled statistical mechanism to address this issue by converting uncalibrated model scores into prediction sets [15], [16]. Rather than trusting raw VLM confidences, CP uses a held-out calibration set to compute thresholds that control the error rate at a user-specified risk level. Under the assumption of exchangeability, this ensures that the probability of excluding the correct action is bounded by the calibration threshold, independent of the VLM architecture or training process.

Building on this principle, we introduce CPNAV, illustrated in Figure 1, a conformal prediction-augmented navigation framework that integrates semantic reasoning with statistically calibrated decision-making. The main contributions of this work are as follows: (1) We introduce a conformal calibration layer that converts uncalibrated VLM action scores into statistically valid prediction sets, guaranteeing under exchangeability that an oracle-consistent trajectory is retained with probability at least $1 - \alpha$, where α is the

¹Qizhao Chen, Shoh Nishino, and Anushri Dixit are with the Department of Mechanical and Aerospace Engineering, University of California, Los Angeles, Los Angeles, CA, USA. qizhaoc@ucla.edu, shohnishino@ucla.edu, anushridixit@ucla.edu

²Yuanhong Zeng is with the Department of Electrical and Computer Engineering, University of California, Los Angeles, Los Angeles, CA, USA. yuanhongzeng@ucla.edu

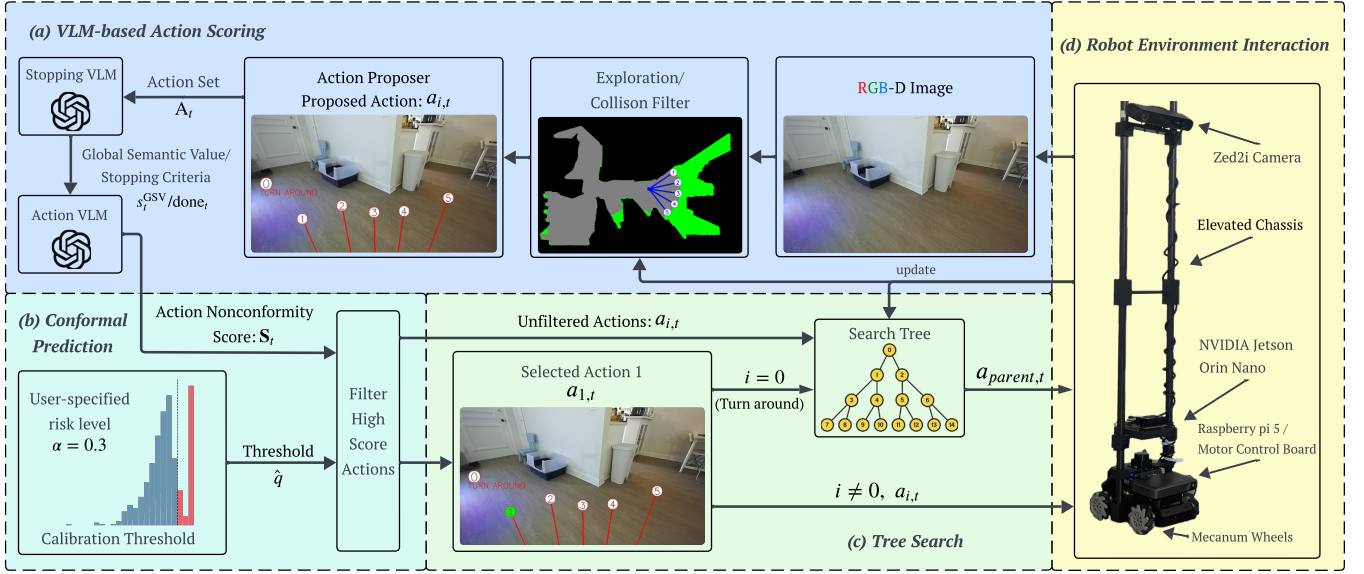


Fig. 1: Overview of the CPNAV pipeline. (a) The action proposer constructs candidate actions from RGB-D input, prioritizing unexplored regions while enforcing collision safety. (b) The threshold obtained via offline conformal calibration is used to filter out actions whose scores fall below the calibrated bound. (c) The highest-scoring untried action that satisfies the threshold is selected and executed, while the resulting transition is recorded in the exploration tree. (d) Hardware experiments are conducted on a modified Hiwonder MentorPi robot platform.

user-specified risk level. (2) An offline oracle-path search is formulated to identify oracle-aligned trajectories by maximizing the minimum VLM action score along a successful path, enabling trajectory-level conformal calibration for long-horizon embodied navigation. (3) Calibrated prediction sets are integrated into a tree-structured exploration policy, where calibrated VLM scores guide action selection and the stored tree supports backtracking.

II. RELATED WORK

A. VLM-Augmented Vision-Language Navigation

Recent research has shifted from reinforcement learning toward leveraging the zero-shot reasoning of VLMs for embodied navigation [1], [9], [10], [17]. Recent advancements in VLM-augmented navigation have expanded spatial reasoning by using large-scale pre-trained models for end-to-end navigational question answering and spatial reasoning [18]–[21], as well as hierarchical world-model and long-horizon planning [22]–[24]. These frameworks leverage diverse strategies such as task-preferenced grounding for flexible objective specification [22] and zero-shot semantic localization that merges scene priors with frontier-based exploration policies [5], [10], [25]. Despite these gains, a persistent challenge remains: these methods often treat VLM outputs as absolute truths. As highlighted in recent literature, this leads to catastrophic failure modes where agents are misled by hallucinated semantic relevance or overconfident scores for physically inaccessible paths [12], [13]. This vulnerability underscores the critical need for a formal statistical mechanism to quantify and calibrate VLM uncertainty during sequential decision-making.

B. Uncertainty Quantification for Perception and Planning

Uncertainty in learned perception and planning modules is commonly quantified with Bayesian neural networks (BNNs) and their approximations such as Monte Carlo dropout [26], with deep ensembles [27], or with post-hoc calibration of softmax confidences [28]. However, BNNs and ensembles require access to model weights and multiple forward passes or multiple trained models, incurring substantial training and inference cost [27], [28], and their confidences remain heuristic without finite-sample guarantees. These methods are also tied to a specific architecture and training procedure, so uncertainty estimation for large language and vision-language models typically falls back on sampled or verbalized outputs [14], [29]. Formal guarantees have instead come from probably approximately correct (PAC) techniques, such as PAC-Bayes bounds on policy generalization [30] and PAC confidence sets calibrated on held-out data [31], or from robust control that bounds a learned perception map’s error near its training data under smoothness assumptions [32], which respectively require policy training with a good prior or assume known linear dynamics. Conformal prediction avoids these limitations: it is model-agnostic and can be readily applied to any pretrained model or policy without re-training, access to model weights, or multiple forward passes, requiring only a held-out calibration set to convert heuristic scores into prediction sets with distribution-free coverage guarantees [16], [33], and with a finite-sample correction it provides the same PAC-style training-conditional guarantee [34].

C. Conformal Prediction for Reliable Decision-Making

Conformal prediction (CP) provides a distribution-free framework for uncertainty quantification with finite-sample coverage guarantees [15], [16]. By converting model scores into calibrated prediction sets, CP bounds the probability of excluding the correct label at a user-specified risk level, making it appealing for safety-critical robotics applications.

Building on these theoretical foundations, several recent works have successfully integrated conformal prediction into the robotics pipeline to enhance safety and reliability. CP has been utilized in perception and object detection to transform standard classification outputs into class prediction sets, preventing autonomous systems from making overconfident but incorrect categorical decisions [33]. Beyond perception, CP has been applied to motion prediction and trajectory forecasting, where it constructs uncertainty-aware prediction tubes around learned dynamics models, enabling downstream planners to account for bounded future-state uncertainty [35]–[38].

Despite these successes, a significant gap remains in high-level navigation decision-making for object-goal navigation. Existing research primarily focuses on uncertainty-aware stopping criteria or confidence estimation for downstream tasks, rather than calibrating the sequential action-selection process itself [12], [39], [40]. While such methods determine when an agent should terminate exploration, they do not provide statistical guarantees over which navigation actions should be executed during long-horizon object-goal search. In contrast, our work applies conformal prediction directly to VLM-based action scoring, constructing calibrated prediction sets that govern the navigation policy throughout the exploration process.

III. METHOD

A. Problem Definition

We consider object-goal navigation in previously unseen indoor environments. An embodied agent is initialized at an unknown pose, $\mathbf{x}_0 = (x_0, y_0, \theta_0)$, where $(x_0, y_0) \in \mathbb{R}^2$ denotes the planar position and $\theta_0 \in [-\pi, \pi]$ denotes the robot orientation. Given a target object category g , such as chair, bed, or toilet, the goal is to navigate to any visible instance of that category without prior knowledge of the map or goal location. At each time step t , the agent receives an RGB-D observation $I_t = (I_t^{\text{RGB}}, I_t^{\text{depth}})$ and its pose $\mathbf{x}_t = (x_t, y_t, \theta_t)$. Based on the current observation history, the agent selects a motion command $a_t = (\phi_t, d_t)$, where ϕ_t denotes the commanded heading direction relative to the robot’s current pose and d_t denotes the commanded travel distance. By executing these motion commands over time, the agent aims to reach the goal object efficiently while following semantically reasonable paths.

B. VLM-based Action Scoring

CPNAV proposes collision-free candidate actions from RGB-D input and queries two VLMs: a stopping VLM that decides goal-reaching and computes a global semantic value, and an action VLM that scores each candidate. The

fused per-action scores guide action selection within the tree-structured exploration process and are used by the conformal prediction layer for calibration.

1) **Action proposer:** The action proposer generates a set of collision-aware motion primitives from the current RGB-D observation (Figure 1a). Building upon the strategy of VLMNav [18], we integrate a depth-based collision filter to ensure safe exploration.

First, the robot generates a navigability mask from the depth image I_t^{depth} by identifying locally traversable regions, as shown in Figure 2. A region is considered traversable if its estimated height variation is below 5 cm. The robot then uniformly samples evenly spaced ray directions $\phi_{i,t}$ across its horizontal field of view, where i indexes each direction, and computes the maximum collision-free travel distance as

$$d_{i,t} = \max\{d \in [0, d_{\max}] \mid \phi = \phi_{i,t}\}.$$

Here, $d_{\max}$ is the maximum considered travel distance and $d_{i,t}$ is the farthest collision-free distance along $\phi_{i,t}$. Collision checking uses an inflated footprint of radius $r_{\text{robot}} + r_{\text{clearance}}$ to account for localization and depth-estimation errors.

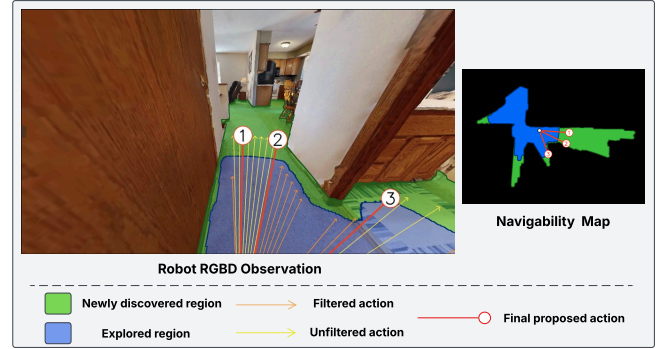


Fig. 2: The action proposer uses the RGB-D observation to identify newly discovered regions and generate collision-free candidate actions toward them.

The resulting distances form a set of candidate motion primitives. To prioritize efficient exploration, we remove primitives shorter than 0.75m, since they provide little progress. Longer collision-free rays are clipped to 1.7m rather than discarded to avoid the robot from making long actions that miss local observations. We further filter actions that lead to previously explored regions by checking whether the endpoint neighborhood of the proposed motion has already been visited in the exploration map. Actions whose endpoints lie mostly in explored regions are removed, encouraging the robot to select motions that expand the explored area. Additionally, we include an auxiliary turn-around action $a_{0,t} = (\pi, 0)$. This action has zero forward displacement and commands an in-place rotation, providing a fallback option when no forward candidate is selected.

The final candidate action set is defined as:

$$A_t = \{a_{i,t} = (\phi_{i,t}, d_{i,t})\}_{i=0}^N, \quad (1)$$

where N denotes the number of feasible motion primitives remaining after collision filtering and exploration prioritization. These candidates are subsequently proposed to the VLM for final selection.

2) **VLM Scoring and Stopping:** At each action step t , the agent queries a VLM to determine if it has found the goal and what action to take to move to the goal.

a) **Goal Reaching Determination and Global Semantic Scoring:** Given the current RGB observation I_t^{RGB} and the object-goal description g , the stopping VLM outputs

$$(\text{done}_t, s_t^{\text{GSV}}) = f_{\text{stop}}(I_t^{\text{RGB}}, g), \quad (2)$$

where $f_{\text{stop}}(\cdot)$ denotes the stopping VLM query (see Figure 3: Stopping Prompt), $\text{done}_t \in \{0, 1\}$ indicates whether the agent has found the goal, i.e., the agent sees the goal, and $s_t^{\text{GSV}} \in [0, 1]$ represents the global semantic value (GSV). GSV measures how promising the current view is for exploration, reflecting the availability of navigable free space. Higher values indicate stronger scene-level cues, and we use GSV to weight per-action confidence scores.

b) **Action Selection and Confidence Scoring:** For each candidate motion primitive $a_{i,t} \in A_t$, the action VLM assigns a semantic confidence score (see Figure 3: Action Prompt),

$$s_{i,t}^{\text{action}} = f_{\text{action}}(I_t^{\text{RGB}}, a_{i,t}, g), \quad (3)$$

where $f_{\text{action}}(\cdot)$ denotes the action VLM query and $s_{i,t}^{\text{action}} \in [0, 1]$ denotes the VLM-estimated semantic score for action $a_{i,t}$ under the current observation. Higher values indicate that the VLM judges the action to be more relevant or promising for reaching the goal.

To incorporate scene-level navigation potential, we fuse the per-action confidence with the global semantic value:

$$s_{i,t} = s_t^{\text{GSV}} \cdot s_{i,t}^{\text{action}}. \quad (4)$$

The final confidence vector used for decision-making at an observation step is

$$\mathbf{S}_t = [s_{0,t}, \dots, s_{N,t}], \quad (5)$$

which collects the scores of all $N + 1$ candidate actions in A_t , including the turn-around action. The fused score $s_{i,t}$ is used to prioritize candidate actions during exploration, offline oracle-path search, and conformal calibration. Concise versions of the prompts and their corresponding JSON response examples are shown in Figure 3.

C. Tree-Structured Exploration with Backtracking

We represent the agent’s exploration history as a dynamically growing tree to support action selection and backtracking. Each node records a visited state, its candidate actions, and their VLM scores, while each edge represents an executed action. At each node, the policy selects the highest-scoring untried admissible action. When no admissible actions remain, the tree enables the agent to return to a previous decision point and consider an alternative action.

Stopping Prompt
 You are a navigation assistant for an agent searching for a {goal_upper}. The input is the agent's current camera view.
STEP 1: VISUAL ANALYSIS
 Describe the visible environment and explicitly state whether a {goal_upper} is present. Strict Identification Rules: A chair is NOT a sofa; a sofa is NOT a bed. Do NOT infer the {goal_upper} from room type. Confirm presence only with high visual confidence.
STEP 2: GOAL VERIFICATION (JSON)
 Determine if the agent has found the {goal_upper}. Output: {"done": <1 or 0>}. Set to 1 ONLY if the {goal_upper} is clearly visible and within 2 meters. Set to 0 if not visible or uncertain.
STEP 3: EXPLORATION POTENTIAL (JSON)
 Independently rate the environment's Global Semantic Score (0.0-1.0) based ONLY on navigability, not on the presence of the {goal_upper}.

Scoring Rubric:
 0.0-0.2: Blocked view, cluttered, no navigable path.
 0.3-0.6: At least one clear outlet or modest navigable space.
 0.7-1.0: High potential; multiple corridors, open doorways, or large open areas.
 Output: {"global_semantic_score": <float>}

Action Prompt
 Choose the path to move close to the nearest {goal_upper}. Use common sense about typical home layouts to guide your search. The image is a view from your current position. Available paths are shown by red arrows labeled 0..{num_actions}. Action 0 (Turn Around): pick if you must backtrack or no visible path leads toward the {goal_upper}. Exploration: prefer directions likely to lead to rooms where a {goal_upper} is typically found.

Reply in three parts:
 1. Observation — describe the scene and any visual leads for the {goal_upper}. 2. Strategy — which general direction is most promising and why. 3. Action (JSON) — exactly:
 {"action": <int: best_action_index>, "score": <float: highest_score>, "confidence_scores": [<score_0>, ..., <score_{num_actions}>]}

Provide exactly {num_actions_plus_one} scores in 'confidence_scores' (Action 0..Action {num_actions}). Each score $\in [0.0, 1.0]$. 'score' = estimated probability (0.0-1.0) that the chosen action will lead toward the {goal_upper}.

Stopping Prompt JSON Response
 "done": 0
 "global_semantic_score": 0.78

Action Prompt JSON Response
 "action": 1
 "score": 0.78,
 "confidence_scores": [0.12, 0.78, 0.52]



Fig. 3: Concise stopping and action prompts with their JSON responses. The stopping prompt, $f_{\text{stop}}(\cdot)$ queries the stopping condition, done_t , and GSV, s_t^{GSV} ; the action prompt, $f_{\text{action}}(\cdot)$, queries the per-action confidence score, $s_{i,t}^{\text{action}}$. The manually normalized confidence score is multiplied by GSV to form the final per-action score, $s_{i,t}$.

The tree serves as an exploration-history structure rather than a conventional planning tree: the policy does not perform rollouts, optimize cumulative path costs, or estimate long-horizon values. Instead, calibrated VLM scores determine action priority, while the tree records alternative actions and supports local and global backtracking.

1) **Exploration Tree Representation:** The agent’s exploration history is represented as a dynamically growing tree. Each node corresponds to a visited state of the robot, and each edge represents an executed action that connects a parent state to a newly reached state. Branching occurs when multiple candidate actions are available from the same state.

Let k denote the current time step. The robot maintains an exploration log,

$$\mathbf{H}_k = \{(\mathbf{x}_t, A_t, \mathbf{S}_t)\}_{t=1}^k, \quad (6)$$

which stores the state, candidate actions, and fused scores of every visited step $t \leq k$. When the agent reaches $\mathbf{x}_k$ by executing an action from an earlier state $\mathbf{x}_t$, we assign the parent of node k to be t . The tree structure is defined by the parent links, while $\mathbf{H}_k$ stores the associated data.

2) **Local Backtrack:** If the VLM selects the turn-around action (index 0), the robot reverts to the parent node t^* of the current time step k and retrieves its stored state and action

set. The remaining untried actions are,

$$R_{t^*} = \{a_{i,t^*} \in A_{t^*} \mid a_{i,t^*} \notin T_{t^*}\}. \quad (7)$$

where $T_t \subseteq A_t$ denotes the set of actions already attempted from step t .

If R_{t^*} is nonempty, the remaining actions are sorted by their semantic confidence scores and the highest-ranked untried action is executed from the parent state $\mathbf{x}_{t^*}$. Otherwise, the agent initiates a global backtracking procedure. At the root node of step 0, the turn-around action is executed directly.

3) **Global Backtrack**: If local backtrack exhausts all recent branches, a global backtrack searches the full history up to the current time step k for any node that still contains untried actions:

$$R_k^{\text{global}} = \bigcup_{t=1}^k \{a_{i,t} \in A_t \mid a_{i,t} \notin T_t, i \neq 0\}. \quad (8)$$

If R_k^{global} is empty, exploration terminates. Otherwise, the action with the highest semantic score among R_k^{global} is selected. The agent is restored to the corresponding state $\mathbf{x}_t$ and resumes exploration by executing that action.

D. Offline Oracle-Path Search

During the offline calibration procedure, once the goal is reached for the first time, we build a candidate path to the goal. However, the initial candidate path may be circuitous or include low-confidence actions. To find the highest-confidence goal-reaching path and avoid overly conservative calibration, we explore the remaining untried actions for paths with higher overall confidence. In the exploration tree, nodes are visited states $\mathbf{x}_t$ and edges are executed actions $a_{i,t}$, each carrying its fused score $s_{i,t}$. Let P denote a path from the root to a goal-reaching node, i.e., the sequence of executed actions along it. The oracle path P^* is the path whose lowest action score is largest:

$$P^* = \arg \max_P \left(\min_{a_{i,t} \in P} s_{i,t} \right). \quad (9)$$

We call this minimum score along a path its bottleneck, denoted $b(P)$. Because calibration is performed over whole trajectories and the prediction set retains a trajectory exactly when it retains its lowest-scoring action, the bottleneck $b(P^*)$ of the oracle path is the trajectory-level confidence used for calibration instead of a per-step guarantee. We implement this via a search algorithm modified from Breadth-First Search, which prioritizes paths with higher bottleneck scores. The resulting path P^* provides the minimum confidence threshold experienced during a successful goal-reaching mission.

To further optimize the oracle path, the agent re-enters a global backtracking phase where exploration is constrained to untried actions $a_{i,t}$ with scores strictly exceeding the current bottleneck $b(P^*)$:

$$R_k^{\text{refine}} = \{a_{i,t} \in R_k^{\text{global}} \mid s_{i,t} > b(P^*)\}. \quad (10)$$

The agent continues this search until it reaches the goal again, triggers a turn-around, or converges to an existing node. In any such event, the search algorithm is re-executed until the total step limit is met. If a higher bottleneck score is found, P^* is updated. This iterative refinement ensures the final bottleneck $b(P^*)$ used for conformal calibration represents the most reliable path discovered.

E. Offline Calibration with Conformal Prediction

To provide rigorous guarantees in unseen environments, we employ a calibration framework that aligns the VLM's internal confidence with actual success rates. This process ensures that the robot is aware of its own uncertainty and only executes actions when they fall within a statistically backed prediction set.

1) **Non-conformity Scoring**: We begin by establishing an offline calibration dataset consisting of $n = 500$ independent and identically distributed episodes sampled using a fixed random seed, where each episode contains a randomly sampled scene, initial pose, object goal, and the corresponding oracle-action trajectory. For each scenario, we compute a non-conformity score E_j , which quantifies the model's trajectory-level error using the minimum oracle-action score along the oracle trajectory identified during our offline oracle-path search:

$$E_j = 1 - \min_t s_{a^*,t,j}, \quad (11)$$

where $s_{a^*,t,j}$ is the semantic confidence score assigned by the VLM to the oracle action a^* , the action along P^* , at time step t of trajectory j .

The VLM is tasked with giving unnormalized scores, which reduces bias. We then normalize the scores for cross-step consistency. High E_j values indicate that the model was "surprised" by the correct action, while low values indicate high alignment.

2) **Beta-Calibration for Uncertainty Alignment**: Because the calibration set is finite, the empirical quantile alone may not guarantee the target coverage $1 - \alpha$ on new data. We therefore use a failure probability $\delta = 0.01$ and define a stricter adjusted risk level $\hat{\alpha}$ such that

$$\text{Beta}^{-1}(\delta; n + 1 - v, v) \geq 1 - \alpha, \quad (12)$$

where $v = \lfloor (n + 1)\hat{\alpha} \rfloor$ and $\text{Beta}^{-1}(\delta; a, b)$ is the δ -quantile of the $\text{Beta}(a, b)$ distribution. This ensures, with confidence $1 - \delta$ over the calibration set, that the oracle-consistent trajectory is retained with probability at least $1 - \alpha$ on a new exchangeable episode.

3) **Quantile Application**: Using the adjusted risk $\hat{\alpha}$, we compute the calibrated threshold $\hat{q}$ as the conformal quantile of the non-conformity scores [15], [16]:

$$\hat{q} = \text{Quantile} \left(\{E_1, \dots, E_n\}, \frac{\lceil (n + 1)(1 - \hat{\alpha}) \rceil}{n} \right) \quad (13)$$

The calibrated quantile $\hat{q}$ is then used to construct the admissible prediction set during online deployment.

F. Online Deployment with Calibrated Prediction Sets

During online inference, the agent uses the offline-calibrated threshold $\hat{q}$ to manage uncertainty. Instead of a greedy execution of the highest VLM-scored action, the agent filters candidate actions into a prediction set to maintain the success guarantee.

1) **Prediction Set Construction:** At each time step t , the agent receives candidate actions A_t and their semantic scores S_t . It constructs a prediction set $C(\xi_t)$, where $\xi_t = (I_t^{\text{RGB}}, g, A_t)$ denotes the query context at step t , containing only actions that satisfy the calibrated threshold:

$$C(\xi_t) = \{a_{i,t} \in A_t \mid s_{i,t} \geq 1 - \hat{q}\} \quad (14)$$

If $C(\xi_t) = \emptyset$, the agent triggers the turn-around action $a_{0,t}$ to initiate backtracking. Otherwise, the agent selects the action with the highest semantic score within $C(\xi_t)$ for execution.

Let a_t^* denote the oracle action at time t from the offline oracle-path search. Under exchangeability between calibration and deployment episodes, the conformal threshold $\hat{q}$ ensures

$$\mathbb{P}(a_t^* \in C(\xi_t), \forall t > 0) \geq 1 - \alpha \quad (15)$$

Because calibration is performed over complete navigation episodes using the bottleneck oracle path, this guarantee applies at the trajectory level. With probability at least $1 - \alpha$, the oracle-consistent path is not pruned during online exploration. The Beta correction further ensures that this coverage bound holds with confidence $1 - \delta = 0.99$ over the sampling of the finite calibration set. The proof is given in Appendix A.

2) **Score-Guided Tree Exploration:** The calibrated prediction set determines which actions are admissible for execution. Among the admissible actions, the agent selects the highest-scoring untried action and records the resulting transition in the exploration tree. This produces a tree-structured exploration history rather than a conventional rollout- or cost-based planning tree.

If the selected action corresponds to index 0 (the turn-around action), the agent rewinds to its parent node and executes the next untried action with the highest score from that parent state. This same procedure is triggered when all admissible actions along the current branch have been exhausted.

When a branch of the exploration tree contains no remaining untried admissible actions, the agent performs a global backtracking step. It traverses the tree to locate the closest ancestor node that still contains untried admissible actions and selects the highest-scoring action at that node. This procedure uses the stored tree to revisit previous decision points while prioritizing actions according to their calibrated VLM scores.

IV. EXPERIMENTAL SETUP

A. Simulation and Hardware Configuration

We consider two experimental settings: simulation and hardware. In the simulation setting, we evaluate CPNAV

using the Habitat-Sim platform on HM3D v0.2 as our primary benchmark and MP3D as an additional benchmark for assessing cross-dataset generalizability [41]–[43]. The agent uses a camera with a wide horizontal field of view (HFOV) and a maximum budget of 200 steps. For the hardware setting, calibration is performed using a simulator configuration that reflects the physical platform’s constraints, including a reduced camera HFOV and a substantially smaller step budget. We additionally conduct a simulation parameter-sensitivity analysis to assess the robustness of CPNAV to sensing configuration and model capacity. To ensure computational efficiency, we implement a parallelized execution pipeline on an NVIDIA RTX 4090 GPU.

For the hardware validation, we employed a Hiwonder MentorPi M1 [44] mobile platform equipped with a Mecanum drivetrain for omnidirectional mobility. State estimation was performed using the SLAM Toolbox, which fused data from a 2D LiDAR scanner, wheel odometry, and an Inertial Measurement Unit (IMU) to provide robust localization. We used a ZED 2i stereo camera [45] to obtain rectified RGB-D data for environmental perception. The specific experimental parameters and sensor configurations are detailed in Table I.

TABLE I: Experimental Parameters

Parameter	Simulation	Hardware
$r_{\text{robot}}, r_{\text{clearance}}$	0.1 m, 0.05 m	
Resolution	640 × 480 RGB-D	
Camera Height	0.9 m	
HFOV / Down Pitch	131° / 0.45 rad	101° / 0.4 rad
VLM Model	GPT-5 Nano [46]	
Step Limit	200	100
Target Risk (α)	0.3	0.45
Failure Probability (δ)	0.01	
Episodes (Cal/Eval)	500 / 200	500 / 24 (Real-world)

B. Baselines and Metrics

To evaluate the effect of conformal calibration on action selection, we compare CPNAV with alternative strategies that incorporate uncertainty in different ways [12]. We also compare with STRIVE [47], the state-of-the-art object-navigation method, and WMNav [48], which uses macro-actions similar to those in the CPNAV pipeline. Although outperforming specialized navigation methods is not our primary focus, these comparisons contextualize performance and demonstrate the applicability of conformal prediction to embodied navigation.

- **No Calibration:** The agent executes the action with the highest raw VLM score $s_{i,t}$ without uncertainty calibration, following the VLMNav baseline [18];
- **Simple Set:** Actions are ranked by score and added in descending order to form the smallest set whose cumulative score reaches at least $1 - \alpha$;
- **Prompt Set:** The VLM is prompted to directly list all feasible actions that could lead to the goal;
- **Ensemble Set:** Builds on the Simple Set by querying the VLM multiple times and using response frequencies to construct the confidence vector S_t .

For each benchmark, all methods, including STRIVE and WMNav, are evaluated on the same 200 episodes using the same object categories and online random seed. The online seed differs from the calibration seed, ensuring identical scenes, initial poses, and object goals across methods. Because STRIVE and WMNav use slightly different criteria for grading success, we reevaluate them using the same manual verification criterion as CPNAV. An episode is successful only when the correct target is visible in the final observation and can be reached through an unobstructed path. Stopping near an incorrect target is counted as failure, even if the stopping-radius condition is satisfied. We report Success Rate (SR), Success weighted by Path Length (SPL) [49], and mean distance traveled. SPL uses the geodesic distance from the start pose to the verified goal computed through the Habitat API.

For the hardware experiments, we compare CPNAV against the no calibration baseline in two different apartment environments with floor plans shown in Appendix B. Under the baseline, no confidence threshold is applied, and all proposed actions are considered admissible and executed in descending order of their raw VLM scores. For each trial, the agent is randomly initialized within the apartment, and the target object category is randomly selected from the same goal set used in simulation. The baseline is evaluated over 24 trials, matching the number of hardware runs conducted for CPNAV. We also use SR, SPL, and mean distance traveled for evaluation.

V. RESULTS

A. Simulation Results

1) *Offline Calibration Threshold Analysis:* We compute the conformal threshold from the VLM confidence assigned to the oracle action in each calibration episode; these oracle-action scores form the calibration set. Figure 4 shows the empirical distributions for the two regimes (simulation and hardware), with the dashed red line marking the calibrated threshold $\hat{q}$ (Sec. III-E). With a Beta correction using failure probability $\delta = 0.01$, corresponding to confidence $1 - \delta = 0.99$, the resulting nonconformity quantiles are

$$\hat{q}_{\text{sim}} = 0.940, \quad \hat{q}_{\text{real}} = 0.936,$$

The simulation setting uses a target risk of $\alpha = 0.3$ and gives an adjusted risk level of $\hat{\alpha} = 0.25$ with a nonconformity quantile level of $1 - \hat{\alpha} = 0.75$. The hardware setting uses $\alpha = 0.45$ and gives $\hat{\alpha} = 0.39$ with $1 - \hat{\alpha} = 0.61$.

Two observations follow. First, oracle-action scores are positively skewed and frequently modest, indicating that even correct actions do not always receive high raw VLM confidence in unseen environments. Second, because calibration is performed in the nonconformity space, larger values of E_j correspond to lower semantic confidence. The calibrated threshold $\hat{q}$ therefore lies in the upper tail of the E_j distribution, meaning that conformal prediction removes only the highest nonconformity (i.e., lowest-confidence) fraction of oracle-aligned actions while retaining the majority of reliable decisions. During online deployment, the calibrated

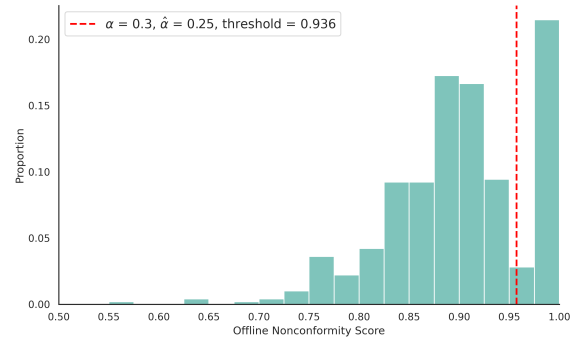


Fig. 4: Empirical distribution of nonconformity scores E_j used for simulation conformal calibration. The dashed red line indicates the calibrated nonconformity quantile $\hat{q} = 0.940$, evaluated at the Beta-adjusted quantile level $1 - \hat{\alpha}$.

TABLE II: Simulation evaluation on HM3D and MP3D (200 episodes per benchmark). Distance is reported in meters.

Policy	HM3D			MP3D		
	SR (%) $\uparrow$	SPL (%) $\uparrow$	Dist. (m)	SR (%) $\uparrow$	SPL (%) $\uparrow$	Dist. (m)
No Calibration [18]	81.0	40.3	35.4	68.5	43.0	36.4
Simple Set	76.7	39.5	32.8	71.0	44.3	33.4
Prompt Set	79.7	38.7	37.4	71.5	45.3	31.6
Ensemble Set	74.0	39.1	28.0	66.5	41.8	28.2
STRIVE [47]	82.5	41.5	17.6	71.5	42.3	21.0
WMNav [48]	80.0	39.9	18.2	75.0	45.2	18.5
CPNAV (Ours)	86.0	44.9	29.2	77.0	46.8	32.1

threshold $\hat{q}$ determines the admissible action set through the prediction-set construction defined in Eq. (14), thereby filtering out low-confidence actions from execution.

2) *Simulation Results:* Table II reports the online evaluation results in the simulation setting. The CPNAV policy uses the offline-calibrated quantile $\hat{q}_{\text{sim}} = 0.94$ to construct admissible action sets at every decision step, which corresponds to a risk level of $\alpha = 0.3$.

Among the calibration baselines, CPNAV achieves the highest SR of 86.0%. Relative to No Calibration, it improves SR by 6.2% and SPL by 11.4% while reducing mean travel distance by 17.5%. It also outperforms the best heuristic threshold-based variant, which achieves 76.7% SR and 39.5% SPL, by 12.1% in relative SR and 13.7% in relative SPL. These gains reflect the intended effect of conformal calibration. Filtering low-confidence branches reduces low-value detours and preserves more of the fixed step budget for promising actions, providing more reliable action selection than fixed heuristic thresholds. Because every episode is capped, steps spent on low-confidence branches cannot be recovered later, so pruning them raises the fraction of episodes that reach the goal before the cap and improves SR and SPL together.

Under each method’s original maximum step limit, CPNAV slightly improves SR and SPL over STRIVE and WMNav. However, the methods use different action representations and step definitions, making their step limits difficult to normalize. However, compared with CPNAV, STRIVE

travels 39.7% and 34.6% shorter mean distances on HM3D and MP3D, respectively, while WMNav travels 37.7% and 42.4% shorter distances. Although CPNAV achieves higher aggregate SPL, its longer mean distance shows that it requires more exploration to obtain this higher success rate. Complete backtracking is a key condition of CPNAV’s theoretical goal-reaching guarantee because it ensures that admissible branches retained by the conformal prediction set are eventually explored when earlier choices fail. However, this can produce additional travel when the goal lies in a lower-ranked or difficult-to-locate region. Thus, CPNAV prioritizes search coverage and reliability over direct travel efficiency, revealing an important tradeoff of its backtracking strategy. We also emphasize that the guarantees provided by our conformal prediction-based framework, CPNAV, can be applied to any embodied navigation framework like WMNav or STRIVE. Appendix C reports CPNAV under reduced step budgets, where it retains its advantage over the uncalibrated baseline, and Appendix D compares the token consumption of all methods.

B. Simulation Parameter Sensitivity Analysis

We analyze the sensitivity of CPNAV to sensing configuration and model capacity by varying one parameter at a time from the CPNAV setting, which is using GPT-5o-nano, height = 0.9 m, HFoV = 131°, calibrated threshold $\hat{q} = 0.940$, and voxel map is enabled to encourage exploration. Table III reports Success Rate, SPL, and mean travel distance.

TABLE III: Parameter sensitivity analysis in simulation.

Variant	SR (%)	SPL (%)	Mean Distance (m)
CPNAV ($\hat{q} = 0.940$)	86.0	44.9	29.2
No Calibration	81.0	40.3	35.4
Height = 0.45 m	67.0	31.4	44.6
Height = 1.70 m	79.3	40.5	30.4
HFoV = 69°	40.0	18.7	44.8
HFoV = 101°	72.7	30.5	40.9
Model = GPT-4o-mini [8]	71.3	33.7	34.4
No Voxel Map	66.0	36.4	36.5

The table highlights four main trends. First, sensing geometry strongly influences performance. Lower camera height or a narrower field of view substantially reduces SR and SPL while increasing travel distance, reflecting less informative observations and less efficient exploration. Second, model capacity matters. Replacing GPT-5o-nano with a smaller VLM degrades both reliability and path efficiency, indicating that calibration effectiveness depends on the semantic strength of the underlying model. Third, conformal calibration consistently outperforms the uncalibrated variant by filtering low-confidence actions and thereby improving both success rate and trajectory efficiency. Finally, maintaining a voxel-based occupancy map further enhances performance by preventing redundant revisits.

C. Hardware Results

We evaluated CPNAV on a physical Hiwonder MentorPi M1 platform across 24 randomized object-goal episodes in two residential apartment layouts. For hardware calibration we used the threshold $\hat{q}$ of 0.936, corresponding to a target risk level of $\alpha = 0.45$. Table IV summarizes the primary metrics comparing the uncalibrated baseline to CPNAV with the calibrated threshold.

TABLE IV: Hardware results.

Policy	SR (%)	SPL (%)	Mean Distance (m)
No Calibration	66.7	42.0	9.0
CPNAV	83.3	48.4	6.3

CPNAV substantially improved reliability and efficiency on the physical robot. The Success Rate increased from 66.7% to 83.3%, corresponding to a relative improvement of 24.9%. Path efficiency improved with a relative SPL increase of 15.2%, while mean distance traveled decreased with a relative reduction of 30.0%. In both simulation and hardware experiments, CPNAV consistently increased Success Rate and SPL while reducing unnecessary exploration. Under the reduced maximum step count and real-world execution noise of the hardware trials, these gains were more pronounced. Conformal calibration systematically filters low-confidence actions that are often semantically plausible but suboptimal, producing more direct trajectories and improved robustness to sensing and actuation uncertainty.

Differences between simulation and hardware results stem from deliberate configuration changes and environment complexity. The hardware experiments used a rectified HFoV of 101°, whereas 131° in simulation, and a modified camera pitch. This reduces visible semantic context and alters viewing geometry. In addition, state-estimation drift during long-horizon backtracking can degrade voxel-map consistency in the real system. Finally, the physical apartments tested were structurally simpler and smaller than many HM3D scenes, which helps explain the higher hardware success rates despite more constrained sensing and noisier odometry.

VI. CONCLUSION AND LIMITATIONS

Conclusion. In this work, we proposed CPNAV, a conformal prediction-based calibration framework for vision-language navigation. By filtering low-confidence actions through data-driven conformal calibration, CPNAV improves both reliability and path efficiency without modifying the underlying policy architecture. Across simulation and real-world experiments, CPNAV consistently achieves higher Success Rate and SPL while reducing unnecessary exploration and travel distance. The improvements are particularly pronounced in hardware deployments, where strict step budgets and real-world execution noise amplify the cost of suboptimal decisions. These results demonstrate that conformal calibration provides a principled and effective mechanism for action filtering.

Limitations. Despite its effectiveness, several limitations remain. First, during offline calibration with the tree-

structured exploration procedure, nodes are defined solely based on spatial proximity without accounting for orientation, which may introduce poor estimation. Second, the method relies on scores from a vision-language model accessed via the ChatGPT API; its inherent stochasticity and occasional inconsistencies can introduce variability in action scoring and affect calibration stability. Finally, direct cross-method benchmarking is not the primary focus of this work, as our contribution centers on conformal calibration for navigation. We nevertheless compare with state-of-the-art navigation methods under a unified evaluation protocol wherever possible, although intrinsic differences in action representations and step definitions prevent complete normalization. Future work will explore orientation-aware state representations, improved robustness to VLM variability, and the integration of our conformal calibration framework with other navigation policies that produce uncalibrated action scores.

REFERENCES

- [1] R. Alqobali, M. Alshmrani, R. Alnasser, A. Rashidi, T. Alhmiedat, and O. M. Alia, "A survey on robot semantic navigation systems for indoor environments," *Applied Sciences*, vol. 14, no. 1, 2023.
- [2] C. Cadena *et al.*, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016.
- [3] G. Dulac-Arnold, D. Mankowitz, and T. Hester, "Challenges of real-world reinforcement learning," *arXiv preprint arXiv:1904.12901*, 2019.
- [4] E. Wijmans, A. Kadian, A. Morcos, S. Lee, I. Essa, D. Parikh, M. Savva, and D. Batra, "Dd-ppo: Learning near-perfect pointgoal navigators from 2.5 billion frames," in *International Conference on Learning Representations (ICLR)*, 2020.
- [5] D. S. Chaplot, D. Gandhi, A. Gupta, and R. Salakhutdinov, "Object goal navigation using goal-oriented semantic exploration," in *Neural Information Processing Systems (NeurIPS)*, 2020.
- [6] Gemini Team, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.
- [7] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [8] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [9] H. Ha and S. Song, "Semantic abstraction: Open-world 3d scene understanding from 2d vision-language models," in *Conference on Robot Learning (CoRL)*, 2023.
- [10] S. Y. Gadre, M. Wortsman, G. Ilharco, L. Schmidt, and S. Song, "CoWs on Pasture: Baselines and benchmarks for language-driven zero-shot object navigation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 23 171–23 181.
- [11] D. Hendrycks, S. Basart *et al.*, "The many faces of robustness: A critical analysis of out-of-distribution generalization," *ICCV*, 2021.
- [12] A. Z. Ren *et al.*, "Robots that ask for help: Uncertainty alignment for large language model planners," in *Proceedings of the Conference on Robot Learning (CoRL)*, 2023.
- [13] M. Ahn *et al.*, "Do as i can, not as i say: Grounding language models in robotic affordances," in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, 2022.
- [14] S. Kadavath *et al.*, "Language models (mostly) know what they know," *arXiv preprint arXiv:2207.05221*, 2022.
- [15] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*, ser. Springer Series in Statistics. Springer, 2005.
- [16] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023.
- [17] Z. Chu, S. Xie, X. Wu, Y. Shen *et al.*, "ABot-N0: Technical report on the VLA foundation model for versatile embodied navigation," *arXiv preprint arXiv:2602.11598*, 2026.
- [18] D. Goetting, H. G. Singh, and A. Loquercio, "End-to-end navigation with vision-language models: Transforming spatial reasoning into question-answering," in *arXiv preprint arXiv:2411.05755*, 2024.
- [19] D. Shah, B. Osinski, B. Ichter, and S. Levine, "Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action," in *Conference on Robot Learning (CoRL)*, 2023.
- [20] G. Zhou, Y. Hong, and Q. Wu, "Navgpt: Explicit reasoning in vision-and-language navigation with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024, pp. 7641–7649.
- [21] G. Zhou, Y. Hong, Z. Wang, X. E. Wang, and Q. Wu, "Navgpt-2: Unleashing navigational reasoning capability for large vision-language models," *arXiv preprint arXiv:2407.12366*, 2024.
- [22] S. Li, D. Huang, Y. He, Y. Fu, Y.-G. Jiang, and X. Xue, "Tp-mddn: Task-preferenced multi-demand-driven navigation with autonomous decision-making," in *Proceedings of the Neural Information Processing Systems (NeurIPS)*, 2025.
- [23] L. Zhang *et al.*, "Mem2ego: Empowering vision-language models with global-to-ego memory for long-horizon embodied navigation," *arXiv preprint arXiv:2502.14254*, 2025.
- [24] Y. Cai, X. He, M. Wang, H. Guo, W.-Y. Yau, and C. Lv, "Cl-cotnav: Closed-loop hierarchical chain-of-thought for zero-shot object-goal navigation with vision-language models," *arXiv preprint arXiv:2504.09000*, 2025.
- [25] N. Yokoyama, S. Ha, D. Batra, J. Wang, and B. Bucher, "VLFM: vision-language frontier maps for zero-shot semantic navigation," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 42–48.
- [26] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proceedings of the International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 48, 2016, pp. 1050–1059.
- [27] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 6402–6413.
- [28] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher, M. Shahzad, W. Yang, R. Bamler, and X. X. Zhu, "A survey of uncertainty in deep neural networks," *Artificial Intelligence Review*, vol. 56, no. S1, pp. 1513–1589, 2023.
- [29] O. Shorinwa, Z. Mei, J. Lidard, A. Z. Ren, and A. Majumdar, "A survey on uncertainty quantification of large language models: Taxonomy, open research challenges, and future directions," *ACM Computing Surveys*, vol. 58, no. 3, pp. 1–38, 2025.
- [30] A. Majumdar, A. Farid, and A. Sonar, "PAC-Bayes control: Learning policies that provably generalize to novel environments," *The International Journal of Robotics Research*, vol. 40, no. 2–3, pp. 574–593, 2021.
- [31] S. Park, O. Bastani, N. Matni, and I. Lee, "PAC confidence sets for deep neural networks via calibrated prediction," in *International Conference on Learning Representations (ICLR)*, 2020.
- [32] S. Dean, N. Matni, B. Recht, and V. Ye, "Robust guarantees for perception-based control," in *Proceedings of the Conference on Learning for Dynamics and Control (L4DC)*, ser. PMLR, vol. 120, 2020, pp. 350–360.
- [33] A. Dixit, Z. Mei, M. Booker, M. Storey-Matsutani, A. Z. Ren, and A. Majumdar, "Perceive with confidence: Statistical safety assurances for navigation with learning-based perception," in *Proceedings of the Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 270, 2025, pp. 2517–2541.
- [34] V. Vovk, "Conditional validity of inductive conformal predictors," *Machine Learning*, vol. 92, no. 2–3, pp. 349–376, 2013.
- [35] L. Lindemann, M. Cleaveland, G. Shim, and G. J. Pappas, "Safe planning in dynamic environments using conformal prediction," *IEEE Robotics and Automation Letters*, vol. 8, no. 8, pp. 5116–5123, 2023.
- [36] A. Dixit, L. Lindemann, S. X. Wei, M. Cleaveland, G. J. Pappas, and J. W. Burdick, "Adaptive conformal prediction for motion planning among dynamic agents," in *Proceedings of the Conference on Learning for Dynamics and Control (L4DC)*, ser. PMLR, vol. 211, 2023, pp. 300–314.

- [37] S. Sheng, P. Yu, D. Parker, M. Kwiatkowska, and L. Feng, "Safe POMDP online planning among dynamic agents via adaptive conformal prediction," *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 9946–9953, 2024.
- [38] A. Srinivasan, A. Leeman, and G. Chou, "Safety beyond the training data: Robust out-of-distribution MPC via conformalized system level synthesis," *arXiv preprint arXiv:2602.12047*, 2026.
- [39] A. Z. Ren, J. Clark, A. Dixit, M. Itkina, A. Majumdar, and D. Sadigh, "Explore until confident: Efficient exploration for embodied question answering," in *Robotics: Science and Systems (RSS)*, 2024.
- [40] K. Liang, Z. Zhang, and J. F. Fisac, "Introspective planning: Aligning robots' uncertainty with inherent task ambiguity," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 71 998–72 031.
- [41] M. Savva *et al.*, "Habitat: A platform for embodied ai research," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 9339–9347.
- [42] K. Yadav *et al.*, "Habitat-matterport 3d semantics dataset," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 4927–4936.
- [43] A. Chang, A. Dai, T. Funkhouser, M. Halber, M. Niessner, M. Savva, S. Song, A. Zeng, and Y. Zhang, "Matterport3d: Learning from RGB-D data in indoor environments," in *International Conference on 3D Vision (3DV)*, 2017.
- [44] Hiwonder, "MentorPi M1 Raspberry Pi 5 ROS2 Robot Car," accessed: Sep. 11, 2026. [Online]. Available: <https://www.hiwonder.com/products/mentorpi-m1-monocular-camera-version>
- [45] Stereolabs, "ZED 2i Stereo Camera," accessed: Sep. 11, 2026. [Online]. Available: <https://www.stereolabs.com/en-us/store/products/zed-2i>
- [46] OpenAI, "OpenAI GPT-5 system card," *arXiv preprint arXiv:2601.03267*, 2025.
- [47] H. Zhu, Z. Li, Z. Liu, W. Wang, J. Zhang, J. Francis, and J. Oh, "STRIVE: Structured representation integrating VLM reasoning for efficient object navigation," *arXiv preprint arXiv:2505.06729*, 2025.
- [48] D. Nie, X. Guo, Y. Duan, R. Zhang, and L. Chen, "WMNav: Integrating vision-language models into world models for object goal navigation," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 2392–2399.
- [49] P. Anderson, A. X. Chang, D. S. Chaplot, A. Dosovitskiy, S. Gupta, V. Koltun, J. Kosecka, J. Malik, R. Mottaghi, M. Savva, and A. R. Zamir, "On evaluation of embodied navigation agents," *arXiv preprint arXiv:1807.06757*, 2018.

APPENDIX

A. Proof of the High-Probability Goal-Reaching Guarantee

Proposition 1. Suppose a goal-reaching oracle trajectory exists within the candidate action structure. Under exchangeability between the calibration and test episodes, complete backtracking over the actions retained by the conformal prediction sets, reliable action execution, and a sufficient step budget, CPNAV reaches the goal with probability at least $1 - \alpha$.

Proof. For calibration trajectory j , define the trajectory-level nonconformity score as

$$E_j = 1 - \min_t s_{a^*,t,j}$$

For a new test episode, define

$$E_{\text{test}} = 1 - \min_t s_{a_t^*,t}^{\text{test}}.$$

By the conformal coverage guarantee,

$$\mathbb{P}(E_{\text{test}} \leq \hat{q}) \geq 1 - \alpha.$$

Whenever $E_{\text{test}} \leq \hat{q}$,

$$1 - \min_t s_{a_t^*,t}^{\text{test}} \leq \hat{q},$$

which implies

$$s_{a_t^*,t}^{\text{test}} \geq 1 - \hat{q}, \quad \forall t.$$

By the definition of the conformal prediction set,

$$C(\xi_t) = \{a_{i,t} \in A_t \mid s_{i,t} \geq 1 - \hat{q}\},$$

every oracle action satisfies

$$a_t^* \in C(\xi_t), \quad \forall t.$$

Therefore, with probability at least $1 - \alpha$, the complete goal-reaching oracle trajectory is not removed by conformal pruning.

The policy may initially execute other admissible actions ranked by their score. Assuming there exists a path to the goal and the step is finite, complete backtracking and a sufficient step budget ensure that every retained untried action is eventually explored. Since all actions in the goal-reaching oracle trajectory P^* remain admissible, the policy eventually traverses P^* . Since the retained oracle trajectory P^* is eventually explored,

$$\mathbb{P}(\text{CPNAV reaches the goal}) \geq \mathbb{P}(E_{\text{test}} \leq \hat{q}) \geq 1 - \alpha.$$

B. Hardware Evaluation Environments

Figure 5 shows the two 1B1B environments used for hardware evaluation. Both are standard apartment settings and are considered in-distribution with respect to the residential environments represented in HM3D and MP3D.



Fig. 5: Floor plans of the two hardware evaluation environments.

C. Performance under Restricted Step Budgets

We evaluate CPNAV on HM3D v0.2 at 50, 100, and 200 steps, corresponding to 25%, 50%, and 100% of the full budget.

TABLE V: Performance under restricted step budgets.

Policy	50 Steps (25%)		100 Steps (50%)		200 Steps (100%)	
	SR (%)	SPL (%)	SR (%)	SPL (%)	SR (%)	SPL (%)
No Calibration	60.3	37.7	75.7	39.8	81.0	40.3
CPNAV	73.0	43.2	80.0	44.4	86.0	44.9

Relative to No Calibration, CPNAV improves SR by 21.1%, 5.7%, and 6.2% and SPL by 14.6%, 11.6%, and 11.4% at 50, 100, and 200 steps, demonstrating stronger performance under limited budgets.

D. VLM Token Consumption

We report the average per-episode input, output, and total VLM token consumption over the 200 MP3D evaluation episodes.

TABLE VI: Average per-episode VLM token usage.

Policy	Input (K)	Output (K)	Total (K)
No Calibration	91.6	139.8	231.4
Simple Set	82.1	125.4	207.5
Prompt Set	81.4	124.3	205.7
Ensemble Set	678.4	1125.0	1803.3
STRIVE	61.3	38.0	99.3
WMNav	91.1	91.0	182.1
CPNAV	79.8	121.8	201.6

CPNAV consumes 12.9% fewer tokens than No Calibration, while Ensemble Set uses approximately 8.9 times more. WMNav and CPNAV both rely heavily on VLM-based reasoning, resulting in comparable token usage. STRIVE uses only 49.3% as many tokens as CPNAV but requires heavier local computation. In our implementation, two concurrent STRIVE instances ran unstably on an RTX 4090, whereas CPNAV supported up to 40 concurrent simulation episodes and ran on the Hiwonder platform without a separate compute server.